# GNOME Extension Manual Installation Guide

This guide provides detailed instructions on how to manually install GNOME Shell extensions from a zip file and how to generate the `gschemas.compiled` file, which is essential for extensions that use GSettings for configuration.

## Table of Contents



---

## 1. Manually Installing a GNOME Extension from a Zip File

GNOME Shell extensions are typically installed via the GNOME Extensions website or through your distribution's package manager. However, sometimes you might need to install an extension manually, for example, if you're developing one, testing a beta version, or if it's not available through official channels.

### Step 1: Locate the Extensions Directory

GNOME Shell extensions are installed in specific directories. The primary location for user-installed extensions is:

```
~/.local/share/gnome-shell/extensions/
```

Each extension resides in its own subdirectory within this path. The subdirectory name must match the extension's UUID (a unique identifier, often found in the extension's `metadata.json` file).

## Step 2: Extract the Zip File

1. **Download the extension's zip file.**

2. **Create the extension's directory:** Navigate to `~/.local/share/gnome-shell/extensions/`. If this directory structure doesn't exist, create it: `bash mkdir -p ~/.local/share/gnome-shell/extensions/`

3. **Identify the Extension UUID:** Before extracting, you need to know the extension's UUID. This is crucial because the extracted folder must be named exactly as the UUID. You can find the UUID inside the `metadata.json` file, which is usually at the root of the zip archive. For example, if the `metadata.json` contains: `json { "uuid": "my-awesome-extension@example.com", "name": "My Awesome Extension", "shell-version": ["42", "43", "44"] }` Your UUID is `my-awesome-extension@example.com`.

4. **Extract the contents:** Extract the *contents* of the zip file into a new directory named after the UUID within `~/.local/share/gnome-shell/extensions/`.

   Let's assume your downloaded zip file is `my-awesome-extension.zip` and its UUID is `my-awesome-extension@example.com`.

   ```
   # Navigate to the extensions directory
   cd ~/.local/share/gnome-shell/extensions/

   # Create a new directory with the extension's UUID
   mkdir my-awesome-extension@example.com

   # Unzip the contents into the newly created directory
   unzip /path/to/my-awesome-extension.zip -d my-awesome-extension@example.com/
   ```

   **Important:** Ensure that the `metadata.json` file and other extension files (like `extension.js`, `stylesheet.css`, etc.) are directly inside the `my-awesome-extension@example.com/` directory, not nested in another subfolder.

## Step 3: Enable the Extension

After placing the extension files, you need to enable it.

1. **Using GNOME Tweaks (Recommended):** If you don't have it, install GNOME Tweaks (also known as "Tweaks" or "GNOME Tweak Tool"): `bash sudo apt install gnome-tweaks` Open GNOME Tweaks, go to the "Extensions" tab, and toggle on your newly installed extension.

2. **Using `gnome-shell-extension-tool` (Command Line - Older GNOME Versions):** For older GNOME Shell versions (prior to GNOME 40), you might use: `bash gnome-shell-extension-tool -e my-awesome-extension@example.com` However, this tool is deprecated.

3. **Using `gsettings` (Command Line - Modern GNOME Versions):** For modern GNOME versions, you can enable extensions using `gsettings`. First, list currently enabled extensions:

   ```
   gsettings get org.gnome.shell enabled-extensions
   ```

   This will output a list like `['extension1@example.com', 'extension2@example.com']`. To enable your new extension, you need to add its UUID to this list.

   ```bash
   # Get current extensions, add yours, and set it back
   CURRENT_EXTENSIONS=$(gsettings get org.gnome.shell enabled-extensions)
   NEW_EXTENSIONS=$(echo "$CURRENT_EXTENSIONS" | sed "s/]$/, 'my-awesome-extension@example.com']/")
   gsettings set org.gnome.shell enabled-extensions "$NEW_EXTENSIONS"
   ```

   Replace `my-awesome-extension@example.com` with your actual extension's UUID.

## Step 4: Restart GNOME Shell (If Necessary)

Sometimes, GNOME Shell needs to be restarted for the new extension to be recognized and loaded.

- **For Xorg sessions:** Press `Alt + F2`, type `r` (or `restart`), and press `Enter`.
- **For Wayland sessions:** You usually need to log out and log back in.

---

# 2. Manually Generating `gschemas.compiled`

Many GNOME applications and extensions use GSettings for configuration. GSettings schemas define the structure and types of configuration options. For an extension to use its own GSettings schemas, these schemas must be compiled into a binary file named `gschemas.compiled`.

## What is `gschemas.compiled`?

`gschemas.compiled` is a binary file generated from XML schema definitions (`.gschema.xml` files). It allows GSettings to efficiently read and apply configuration settings defined by the extension. If an extension uses GSettings and this file is missing or outdated, the extension might not function correctly or its preferences dialog might not appear.

## Step 1: Identify Schema Files

Your extension's GSettings schema definitions are typically located in a `schemas/` subdirectory within the extension's main folder. These files have a `.gschema.xml` extension.

Example path:

```
~/.local/share/gnome-shell/extensions/my-awesome-extension@example.com/schemas/org.gnome.shell.extensions.my-awesome-extension.gschema.xml
```

## Step 2: Compile the Schemas

You use the `glib-compile-schemas` tool to compile the schema files. This tool is usually part of the `libglib2.0-bin` package.

1. **Navigate to the `schemas/` directory:** `bash cd ~/.local/share/gnome-shell/extensions/my-awesome-extension@example.com/schemas/` Replace `my-awesome-extension@example.com` with your extension's UUID.
2. **Run the compilation command:** `bash glib-compile-schemas .` The `.` (dot) tells `glib-compile-schemas` to compile all `.gschema.xml` files in the current directory and place the `gschemas.compiled` file in the same directory.

If successful, you will see a new file named `gschemas.compiled` in your `schemas/` directory.

### Step 3: Verify and Restart

After generating `gschemas.compiled`, restart GNOME Shell (as described in Step 4 of the installation section) to ensure the system picks up the new compiled schemas. You can verify if the schemas are loaded by trying to access a setting using `gsettings`:

```
gsettings list-keys org.gnome.shell.extensions.my-awesome-extension
```

Replace `org.gnome.shell.extensions.my-awesome-extension` with the schema ID defined in your `.gschema.xml` file (it's usually the `id` attribute of the `<schema>` tag). If it lists keys, your schema is loaded.

---

## 3. Troubleshooting and Tips

- **Check `metadata.json`:** Ensure the `uuid` in `metadata.json` exactly matches the folder name. Also, check the `shell-version` array to ensure it includes your current GNOME Shell version.
- **Permissions:** Make sure your user has read and execute permissions for the extension files and directories.
- **Logs:** If an extension isn't working, check the GNOME Shell logs for errors. You can view them using `journalctl`: `bash journalctl -f -o cat /usr/bin/gnome-shell` Look for lines related to your extension.
- **Dependencies:** Some extensions might have external dependencies. Check the extension's documentation or source code for any required packages.
- **GNOME Shell Version:** Extensions are often sensitive to GNOME Shell version changes. An extension built for an older version might not work on a newer one without updates.
- **Extension Manager:** Consider installing the "Extension Manager" application (available in your software center or via `sudo apt install gnome-shell-extension-manager`). It provides a more user-friendly interface for managing extensions, including enabling/disabling and checking for updates.

This guide should help you confidently manage GNOME Shell extensions manually.